IndicNeuroSym: Neuro-Symbolic Constrained Decoding for Classical Telugu *Dvipada* Poetry Generation

Samvaran K. Rallabandi, Mahesh Emani,
Pradeep Pachipulusu, and **Faviana Noronha**
IIIT Hyderabad
{samvaran.rallabandi, mahesh.emani}@research.iiit.ac.in
{saipradeep.p, faviana.noronha}@students.iiit.ac.in

April 10, 2026

Abstract

We present IndicNeuroSym, a neuro-symbolic pipeline for generating metrically valid classical Telugu *dvipada* couplets. *Dvipada* metre enforces strict prosodic rules governing syllable weight (*guru*/*laghu*), metrical foot sequences (*gaṇa*), second-syllable rhyme (*prāsa*), and caesura alliteration (*yati*). We curate a validated corpus of 27,881 couplets from five classical Telugu literary sources supplemented with LLM-generated synthetic data, achieving 100% prosodic integrity via a custom chandas scanner. We fine-tune Gemma 3 1B with instruction-tuned pairs and design a constrained decoding system consisting of a three-stage Finite State Transducer (FST) pipeline feeding three parallel Non-deterministic Finite Automata (NFAs) that enforce metrical constraints at token-generation time. The LoRA fine-tuned 1B model achieves a 100% average prosodic score versus 25.0% for the untuned base model. We further explore if the same architecture is feasible with the Kannada due to similarity of script.

1 Introduction

Classical Telugu literature contains a rich tradition of structured poetic forms. Among these, *dvipada* (literally "two feet") is one of the oldest metres, forming the backbone of major epics such as *Ranganatha Ramayanamu* (13th century) and *Basavapuranamu*. A *dvipada* couplet consists of exactly two lines (*pādalu*), each composed of four metrical feet (*gaṇalu*): three Indra-class feet followed by one

Surya-class foot. Additional constraints govern second-syllable consonant rhyme (*prāsa*) and alliterative harmony between designated feet within a line (*yati*).

Generating metrically valid classical poetry in Telugu and Kannada presents three core challenges. First, both are highly agglutinative Dravidian languages; standard BPE tokenizers (Sennrich et al., 2016) fragment their syllables(aksharas) in ways that misalign with prosodic syllable boundaries. Second, metrical constraints are active in both traditions: while a valid Telugu dvipada couplet must satisfy gana(moraic), prasa, and yati, a Kannada utsaaha ragale similarly demands strict adherence to prasa and exact gana tablet alignments (such as "3+3+3+3" or "3+3+3+Guru"), alongside the prohibition of iambic (laghu-guru) sequences within any given line(pada). Third, annotated training corpora for these classical Dravidian metrical forms remain exceptionally scarce.

This paper makes four contributions: (1) a curated, rigorously validated corpus of 27,881 dvipada couplets with Telugu and English meanings and an additional 1010 synthetically generated Kannada ragale; (2) an instruction fine-tuning (IFT) dataset and training pipeline; (3) a neuro-symbolic constrained decoding architecture using chained FSTs and parallel NFAs that guarantees metrical validity by construction; and (4) baseline evaluations comparing base and fine-tuned models for Generation and Analysis tasks following the taxonomy proposed by Kranti and Vajjala (2024).

2 Related Work

Constrained text generation. Constrained decoding frameworks enforce formal constraints during token generation without model retraining. Outlines (Willard and Louf, 2023) uses FSMs for JSON and regex constraints; LMQL (Beurer-Kellner et al., 2023) provides a query language for structured generation; and Guidance (Guidance-AI, 2023) implements grammar-constrained decoding via token masking. Our work extends this line to Telugu prosodic metre, where constraints derive from classical chandas treatises.

Multilingual sentence embeddings. Cross-lingual alignment is evaluated using LaBSE (Feng et al., 2022), while intra-lingual poem-to-meaning similarity is assessed via mSBERT (**?**). To provide a specialized comparison for Indic languages, L3Cube-IndicSBERT (Deode et al., 2023) is incorporated into the analysis. Furthermore, the LLM-as-a-Judge evaluation protocol is modeled after the G-Eval (Liu et al., 2023) and GEMBA (Kocmi and Federmann, 2023) metrics.

Table 1: Permitted metrical feet (*gaṇalu*). U = *guru* (heavy), I = *laghu* (light).

Foot	Pattern	Class	Beats
nala	I I I I	Indra	4
naga	I I I U	Indra	5
sala	I I U I	Indra	5
bha	U I I	Indra	4
ra	U I U	Indra	5
ta	U U I	Indra	5
na	I I I	Surya	3
ha / gala	U I	Surya	3

Computational prosody for Indic languages. Prior work on Indic metre analysis has focused primarily on SanskritGoyal et al. (2022); Telugu-specific computational prosody remains largely unexplored. To our knowledge this is the first study to combine formal *dvipada*/*ragale* prosody rules with neural language model generation for Telugu.

3 The *Dvipada* Prosodic Framework

Dvipada metre is governed by a hierarchy of rules operating at the syllable, foot, and line levels. Complete rule tables appear in Appendix A.

Structure. A *dvipada* stanza contains exactly two lines (*pādalu*). Each line comprises four metrical feet (*gaṇalu*) in the fixed sequence: three Indra-class feet followed by one Surya-class foot.

Syllable weight. Each syllable is classified as heavy (*guru*, U) or light (*laghu*, I) according to classical rules from *Chandhodarpanamu* (Chirravuri Srirama Murthy, 1998): long vowels, diphthongs, anusvara, visarga, syllable-final consonants, and position before conjunct clusters yield *guru*; short vowels yield *laghu* by default. Full classification rules, including the special treatment of the vocalic *R* and word-boundary constraints, are in Appendix A.1.

Metrical feet. Table 1 lists the eight permitted foot patterns. Six are Indra-class (positions 1–3 per line) and two are Surya-class (position 4).

***Prāsa* (second-syllable rhyme).** The base consonant of the second syllable of line 1 must match that of line 2, subject to three phonemic equivalence groups: laterals {la, ḷa}, sibilants {śa, sa}, and rhotics {ṛa, ra} (Appendix A.2). *Samanya dvipada* strictly enforces *prāsa*; *manjari dvipada* (used in narrative epics) relaxes it.

***Yati* (caesura alliteration).** Within each line, the first syllable of foot 1 must harmonically match the first syllable of foot 3 under 11 *maitri* equivalence groups (Appendix A.3).

4 The *Utsaha Ragale* Prosodic Framework

Utsaha Ragale metre is governed by a hierarchy of rules operating at the syllable, foot, and line levels. Complete rule tables appear in Appendix A.

Structure. A *Utsaha Ragale* stanza contains exactly two lines (*pādalu*). Each line comprises four metrical feet (*gaṇalu*), each of exactly three syllables, yielding 12 syllables per line.

Syllable weight. Each syllable (*akṣaram*) is classified as heavy (*guru*, U) or light (*laghu*, I). Heavy syllables are triggered by long independent vowels, long dependent vowels (*mātra*), diphthongs, anusvara, visarga, *halant* (virama) endings, and position immediately preceding a conjunct cluster or doubled consonant. Short vowels yield *laghu* by default. Crucially, word boundaries block the conjunct rule. Full classification rules and character sets are detailed in Appendix A.1.

Metrical feet. Table 2 lists the permitted foot patterns. The metre strictly prohibits two-syllable patterns (such as I U) to maintain the triple-syllable rhythmic flow.

Table 2: Permitted metrical feet (*gaṇalu*) in *Utsaha Ragale*. U = *guru* (heavy), I = *laghu* (light).

Pattern	Syllables	Allowed Positions	Beats
I I I	3	1, 2, 3	3
I I U	3	1, 2, 3, 4	4

***Guru* ending.** Both lines must end on a *guru* syllable. This is structurally enforced by a strict constraint on the final foot: position 4 must always be the I I U pattern.

***Ādi Prāsa* (second-syllable rhyme).** The base consonant of the second syllable of line 1 must match that of line 2. The base consonant is defined as the primary consonant character, ignoring vowel marks, conjunct extensions, anusvara, and visarga. While strict evaluation requires an exact match, relaxed evaluation permits matching within two phonemic equivalence groups: laterals {la, ḷa} and sibilants {śa, ṣa, sa} (Appendix A.2).

5 Dataset Construction

5.1 Sources and Curation

Our dataset comprises *dvipada* couplets from five classical Telugu sources, supplemented by 1,012 synthetic *ragale* poems across 100 topics generated using Gemini 3 Pro to mitigate data scarcity. Table 3 reports raw couplet counts, the number passing strict 100% prosodic compliance (*pure*), and purity rates per source.

All candidates were filtered by dwipada and ragale analyzers, which verifies full prosodic compliance on. The high rejection rate for *Palanati Vira Charitra* (91.7%) reflects its use of the *manjari* variant, which relaxes *prāsa*. The final dataset comprises 27,881 couplets in JSON format, each paired with a Telugu prose meaning and an English translation.Kannada ragale follows the same structure.

Source	**Raw Couplets**	**Master (Validated)**	**Purity**
Ranganatha Ramayanamu	26,296	21,828	83.0%
Basavapuranamu	2,454	1,859	75.8%
Dvipada Bhagavatamu	3,157	2,002	63.4%
Palanati Vira Charitra	783	65	8.3%
Srirama Parinayamu	392	374	95.4%
Synthetic (Gemini)	3,496	1,753	50.1%
Total	**36,578**	**27,881**	**76.2%**

Table 3: Dataset sources with raw couplet counts, master validated counts, and prosodic purity rates.

5.2 Dataset Composition

The corpus is 93.7% classical material (26,128 couplets) and 6.3% synthetic (1,753 couplets). Topics span theology, mythology, and devotional poetry, primarily in archaic Telugu register with colloquial augmentations in the synthetic subset.

6 Data Validation

We applied a four-level validation pipeline to the 27,881 assembled couplets.

6.1 Level 1: Prosodic Integrity

Every couplet was verified by the chandas scanner. Pass rate: 100% (27,881/27,881), as the scanner served as the inclusion criterion in Section 5.

6.2 Level 2: Length Ratio Check

We enforce $0.8 \leq |P|/|V| \leq 2.5$, where P is the Telugu prose meaning and V is the verse. Pass rate: 99.1% (27,633/27,881). The bulk of the dataset (88.9%) falls in the 1.0–2.0 band; 248 records (0.9%) fall outside the enforced bounds. The five most extreme outliers have ratios of 5.11, 4.73, 4.71, 4.65, and 4.36.

6.3 Level 3: Semantic Fidelity

Cosine similarity between text pairs is computed using three multilingual sentence embedding models. The primary quality gate is LaBSE Telugu-to-English meaning similarity ≥ 0.65 (Feng et al., 2022). LaBSE excels at cross-lingual alignment and passes 77.1% of records on the Telugu–English meaning pair (mean 0.721), confirming that prose annotations are translation-consistent. mSBERT-mpnet (**?**), strongest at within-language similarity, passes 63.7% on poem-to-Telugu meaning (mean 0.685). L3Cube-IndicSBERT (**?**), an Indic-specialised model, reaches 40.1% on the same pair (mean 0.614), reflecting its domain gap with archaic Telugu. Low poem-to-meaning scores across all models are expected: classical *dwipada* verse uses compressed, elliptical syntax and *samāsa* compounds that diverge structurally from modern prose paraphrases. The scores serve as diagnostic controls, not quality indicators.

Table 4: Semantic fidelity evaluation. Primary gate: LaBSE Telugu–English ≥ 0.65. Full per-pair results appear in Appendix C.

Model	**Pair**	**Pass%**	**Mean**
LaBSE (Feng et al., 2022)	Tel.–Eng. meaning	77.1	0.721
mSBERT-mpnet (**?**)	Poem–Tel. meaning	63.7	0.685
L3Cube-IndicSBERT (Deode et al., 2023)	Poem–Tel. meaning	40.1	0.614

6.4 Level 4: Lexical Diversity

Corpus health is assessed via subword token coverage and word-level literary diversity metrics. Using the Gemma 3 tokenizer, the dataset's 804,671 tokens cover 74.4% of the model's Telugu vocabulary (1,328/1,784 tokens). At the word level, corpus TTR is 0.491 and MATTR is 0.501—below the 0.70–0.80 range typical of modern prose, consistent with formulaic elements inherent to metrical verse. Yule's $K = 3.50$ places the corpus firmly in the literary range ($K < 100$), while Honoré's $H = 6{,}165$ far exceeds the 1,000–2,000 literary baseline, indicating exceptional vocabulary richness. The hapax ratio ($V_1/V = 0.804$) confirms that the majority of word types occur exactly once. No exact duplicates were found; 160 poems in 79 groups are near-duplicates under normalisation.

Metric	**Scope**	**Value**	**Reference**
TTR (V/N)	Corpus	0.491	**?**
MATTR (w=50)	Corpus	0.501	Covington and McFall (2010)
Yule's K	Corpus	3.497	Yule (1944)
Honoré's H	Corpus	6164.7	Honoré (1979)
Hapax ratio (V_1/V)	Corpus	0.804	—
Sichel's S (V_2/V)	Corpus	0.091	Sichel (1975)
Per-poem TTR (mean)	Poem	0.993	—
Per-poem MATTR (w=2, mean)	Poem	1.000	—
Per-poem Yule's K (mean)	Poem	22.74	—
Per-poem Honoré's H (mean)	Poem	2356.4	—
Gemma 3 Telugu coverage	Subword	74.4%	—
Per-poem subword TTR (mean)	Subword	0.897	—
Exact duplicates	Corpus	0	—
Near duplicates	Corpus	160 (79 grp)	—

Table 5: Lexical diversity metrics for the validated Dwipada corpus (27,881 poems, 179,812 tokens, 88,295 types).

6.5 LLM-as-a-Judge Evaluation

A stratified sample of 250 couplets was evaluated by Gemini 3.1 Pro (Vertex AI Batch API) using a five-dimensional rubric (1–5 scale) based on G-Eval (Liu et al., 2023) and GEMBA (Kocmi and Federmann, 2023). Dimensions cover: (1) semantic fidelity, (2) completeness, (3) cultural and contextual accuracy, (4) Telugu linguistic quality, and (5) English linguistic quality. Table 6 reports dimension-level results.

Overall verdicts: 90.4% "Excellent" (226/250), 6.4% "Good", 2.8% "Acceptable", and 0.4% "Poor" (1 sample). Full aggregate score distribution and

Table 6: LLM-as-a-Judge evaluation ($n = 250$). Columns give percentage of samples at each rating. Mean total score: 24.25/25 (97.0%).

Dimension	**Mean**	**5**	**4**	**3**	**≤2**
Semantic Fidelity	4.73	80.0	14.4	4.0	1.6
Completeness	4.76	80.8	15.6	2.8	0.8
Cultural Accuracy	4.89	93.6	2.8	2.4	1.2
Telugu Linguistic	4.98	98.0	1.6	0.4	0.0
English Linguistic	4.90	90.4	8.8	0.8	0.0

Table 7: Cross-level validation summary.

Level	**Description**	**Pass Rate**
1	Prosodic integrity (chandas)	100.0%
2	Length ratio (0.8–2.5)	99.1%
3	LaBSE semantic fidelity	77.1%
4	Lexical diversity (TTR)	0.90 avg
Pass all levels 1–3		68.9%

source breakdown appear in Appendix C. Table 7 summarises all four validation levels.

7 Fine-Tuning Dataset Preparation

The validated 27,881-entry corpus is transformed into instruction-completion pairs for supervised fine-tuning (SFT) of Gemma 3 1B (Gemma Team, 2024) using LoRA (Hu et al., 2022).

Instruction templates. Each sample is assigned one instruction type from four categories: (1) **Generic** ("Write a Telugu *dvipada* couplet following the above rules"), available for all samples; (2) **Work-style** ("Compose a *dvipada* in the style of [source]"), available for non-synthetic sources; (3) **Meaning-conditioned (Telugu)**: "Write a *dvipada* expressing: {Telugu meaning}"; and (4) **Meaning-conditioned (English)**: "Write a *dvipada* expressing: {English meaning}". This diversity trains the model to respond to varied prompt styles in both languages.

Rules injection. Every training sample's input is prefixed with the complete *dvipada* prosodic specification, covering all *gaṇa* patterns, *guru/laghu* rules, *prāsa*,

Table 8: Baseline benchmark results (5 prompts per model). "Valid" denotes fully valid couplets. Avg. = *gaṇa* (40%) + *prāsa* (35%) + *yati* (25%).

Model	**Avg**	**Valid**	**Gaṇa**	**Prāsa**	**Yati**
Gemma-3-1B base	25.0%	0%	32.5%	20.0%	20.0%
Gemma-3-1B LoRA (best)	100%	80%	92.5%	100.0%	90.0%
Gemma-3-4B base	50.0%	20%	52.5%	40.0%	60.0%

and *yati*. This ensures the model has the metrical specification in context during both training and inference.

Dataset splits. The pipeline produces `train.jsonl` (80%), `val.jsonl` (10%), and `test.jsonl` (10%, held out).

8 Baseline Evaluation

We evaluate three model configurations on five generation prompts each (15 total generations), scoring each couplet on a weighted prosodic metric: *gaṇa* (40%), *prāsa* (35%), and *yati* (25%). Table 8 reports results.

The LoRA fine-tuned 1B model substantially outperforms both base models. The 4B base model outperforms the 1B base on all metrics, suggesting that scale partially compensates for the absence of domain-specific training. The base 1B model fails on all five prompts, with particular weakness in *prāsa* matching (20.0%). Detailed per-prompt outputs appear in Appendix D. Section 12 extends this baseline to 1,224 constrained generations under the symbolic enforcer.

9 Constrained Decoding Architecture

To provide formal metrical guarantees beyond what fine-tuning alone achieves, we design a constrained decoding system that intercepts Gemma's token generation and masks out any token violating *dvipada* metre.

9.1 Theoretical Basis

Every *dvipada* constraint operates on fixed positions with finite alphabets, placing the combined constraint language within the regular class:

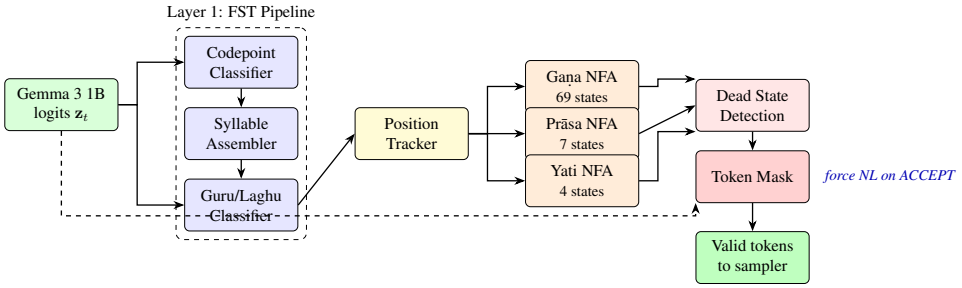

Figure 1: Symbolic Enforcer pipeline. Candidate tokens from Gemma 3 1B pass through a two-stage FST pipeline (with inlined codepoint classifier), are routed by the Position Tracker to three parallel NFAs (Gaṇa, Prāsa, Yati), and receive a boolean mask ($-\infty$ for invalid tokens) via dead state detection. Dashed arrow: raw logits bypass path to the mask.

- **Gaṇa pattern:** The legal syllable sequence per line is a finite union of concatenations of finite strings over $\{\mathrm{U}, \mathrm{I}\}$; no stack or unbounded memory is required.
- **Prāsa:** One stored consonant equivalence class (approximately 30 classes); a single comparison at a fixed position.
- **Yati:** One stored *maitri* group (11 groups); a single comparison at a fixed position.

Since regular languages are closed under intersection and inverse homomorphism, the combined constraint machine remains an NFA of bounded size.

9.2 System Overview

Figure 1 illustrates the Symbolic Enforcer pipeline. Figure 2 shows how the three decoding strategies consume the enforcer's output. At each generation step, raw logits from Gemma 3 1B pass through the Enforcer before sampling.

9.3 Layer 1: Orchestrator FST Pipeline

Two chained Mealy FSTs transform raw Unicode token strings into a classified U/I syllable stream. A stateless character classifier precedes them.

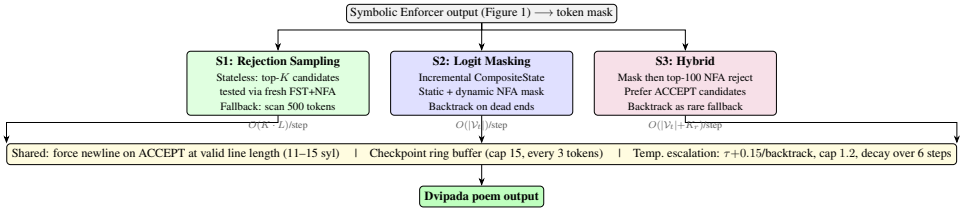

Figure 2: Three constrained decoding strategies using the Symbolic Enforcer. (S1) Rejection sampling re-analyzes full text per candidate (stateless, $O(KL)$). (S2) Logit masking maintains incremental state with dynamic NFA masks ($O(|\mathcal{V}_{\text{tel}}|)$, sequence-length independent). (S3) Hybrid combines masking with rejection on the top-100 masked candidates. S2 and S3 share checkpoint-based backtracking with temperature escalation.

Stage 0: Codepoint Classifier (stateless). A stateless classification function classifies each Unicode codepoint into 9 categories via an $O(1)$ set-membership check. Categories cover consonants (U+0C15–U+0C39), independent vowels (U+0C05–U+0C14), matras (U+0C3E–U+0C4C), virama (U+0C4D), diacritics (anusvara U+0C02, visarga U+0C03), space, newline, skip (ZWNJ, ZWSP, candrabindu), and other. This function is inlined into the Syllable Assembler rather than implemented as a separate transducer.

Stage 1: Syllable Assembler (4 states). Groups the category stream into complete Telugu syllables using states IDLE, CONSONANT_CLUSTER, PENDING_VIRAMA, and VOWEL. A one-slot *prev_syllable* buffer enables retroactive pollu (syllable-final consonant) merging. The PENDING_VIRAMA state distinguishes conjunct consonants from pollu marks via lookahead.

Stage 2: Guru/Laghu Classifier (2 states). Classifies each syllable as U or I using five phonological rules from *Chandhodarpanamu* (Chirravuri Srirama Murthy, 1998), via states EMPTY and PENDING_I. A one-syllable delay buffer in PENDING_I enables Rule 5 (conjunct-induced weight promotion), which applies within words but is blocked by intervening spaces.

9.4 Layer 2: Position Tracker and Parallel NFAs

The Position Tracker routes syllable metadata to three NFAs based on the current position within the couplet: line index (1 or 2), syllable index within the line, and line completion count.

Gaṇa NFA (∼69 states). Validates each line against the regular pattern $(nala \mid naga \mid sala \mid bha \mid ra \mid ta)^3 \cdot (na \mid ha)$ over {U, I}. The NFA branches non-deterministically after each syllable. Each branch is a tuple (`slot`, `gana_name`, `sub_pos`, `matched_ganas`), where slot indexes the current foot (0–2 for Indra ganas, 3 for Surya gana). Valid line lengths range from 11 to 15 syllables.

Prāsa NFA (7 states). Phase 1 records the base consonant of the second syllable of line 1. Phase 2 verifies that the second syllable of line 2 has a matching base consonant (checked via equivalence lookup); a mismatch transitions to the REJECT state. The 7 states are: LINE1_SYL0, LINE1_SYL1, LINE1_REST, LINE2_SYL0, LINE2_SYL1, ACCEPT, and REJECT. Consonant identity is stored as a data variable, not encoded in NFA states. Three equivalence pairs (laterals: l/L, sibilants: sh/Sh/s, rhotics: R/r) are checked via lookup.

Yati NFA (4 states). Phase 1 records the phonetic information of the first syllable of *gaṇa* 1. Phase 2 passes through *gaṇa* 2 silently. Phase 3 verifies the first syllable of *gaṇa* 3 belongs to the same phonetic class via a 5-level cascade (exact match, *vyanjana yati*, *svara yati*, *samyukta yati*, *bindu yati*). The 4 phases are: IDLE (awaiting *gaṇa* 1 first syllable), RECORDED (*gaṇa* 1 info stored, awaiting *gaṇa* 3), ACCEPTED (*yati* check passed), and REJECTED (*yati* check failed). Phonetic group membership (across 11 *Yati Maitri* groups) is computed on demand rather than encoded in NFA states. *Yati* enforcement is optional; it can be disabled for *manjari* variant generation.

9.5 Dead State Detection

To maintain metrical integrity, we define a *Dead State Detection* mechanism. Let $\mathcal{B}$ be the set of active NFA branches. For each branch $b \in \mathcal{B}$ at state $s = (\texttt{slot}, \texttt{gana}, \texttt{pos})$, let $C(b)$ be the current syllable count. We define two closed-form heuristic functions:

- $min_dist(s)$: Minimum syllables to reach ACCEPT from s.
- $max_dist(s)$: Maximum syllables to reach ACCEPT from s.

A branch b is considered **viable** if it satisfies the window constraint $[L_{min}, L_{max}]$, where $L_{min} = 11$ and $L_{max} = 15$:

$$(C(b) + \textit{min_dist}(s) \leq 15) \wedge (C(b) + \textit{max_dist}(s) \geq 11) \tag{1}$$

If the set of viable branches is $\emptyset$, the configuration is flagged as a *dead state*, triggering immediate token pruning.

Component	Type	States
Codepoint Classifier	Stateless	0 (inlined)
Syllable Assembler	Mealy FST	4
Guru/Laghu Classifier	Mealy FST	2
Position Tracker	Controller	$\sim$3 (cntr)
Gaṇa NFA	NFA	69
Prāsa NFA	NFA	7
Yati NFA	NFA	4
Total (separate)		**$\sim$80**
Product (equiv.)	DFA	1,932

Table 9: State budget for the constrained decoding system.

9.6 State Budget and Design Rationale

Table 9 compares the separate-NFA design against the equivalent product automaton. Using three independent NFAs achieves a 25$\times$ state reduction ($60 \times 7 \times 4 = 1{,}680$ states versus $\sim$80), while enabling partial scoring, selective constraint enforcement, and independent debugging per constraint.

9.7 Constrained Decoding Loop

The system implements three decoding strategies. Full pseudocode for all strategies appears in Appendix G.

Strategy 1: Rejection Sampling (Algorithm 4). At each generation step: (1) Gemma produces logits $\mathbf{z}_t$; (2) the top-K candidates (default $K = 50$) are tested sequentially via ANALYZETEXT (Algorithm 2), which runs a fresh FST+NFA pipeline on the full text; (3) tokens causing any NFA to enter a dead or doomed state — detected by ISREACHABLE (Algorithm 1) — are rejected; (4) the first valid candidate is accepted, preferring tokens that reach a line-completion (ACCEPT) state. If no valid candidate is found in top-K, an exhaustive vocabulary search is

performed.

Strategy 2: Logit Masking with Backtracking (Algorithm 5). At each generation step: (1) Gemma produces logits $\mathbf{z}_t$; (2) a static mask eliminates all non-Telugu tokens ($\sim$256K vocabulary reduced to $\sim$1,822 Telugu tokens); (3) BUILDGANAMASK (Algorithm 3) tests each Telugu token against a snapshot of the current COMPOSITESTATE, setting invalid tokens to logit $-\infty$; (4) the surviving distribution is sampled normally. If the valid token count drops below a threshold (default 3) or the line overshoots 15 syllables without reaching ACCEPT, the system backtracks to a saved checkpoint. Checkpoints are stored in a ring buffer (max 15) saved every 3 tokens. On backtrack, the generation temperature is escalated by $+0.15$ (capped at 1.2) and decays back to the base over 6 steps.

Strategy 3: Hybrid Mask + Rejection (Algorithm 6). Combines both approaches: (1) logit masking narrows the vocabulary to $\sim$1,000 valid Telugu tokens; (2) BUILDGANAMASK (Algorithm 3) further prunes invalid tokens; (3) from the top-100 candidates, each is tested via COMPOSITESTATE cloning; (4) candidates reaching line-completion (ACCEPT) are preferred over merely alive candidates; (5) backtracking is triggered only if no valid candidate exists in the top-100.

10 Ragale Constrained Decoding Adaptation

The *Utsaha Ragale* pipeline reuses the full Symbolic Enforcer architecture from Section 9 with two structural simplifications: the Gaṇa NFA is replaced with a simpler 2-pattern variant, and the Yati NFA is removed entirely. All six decoding algorithms in Appendix G apply without modification.

10.1 Structural Differences from Dvipada

Table 10 summarises the key differences. The FST pipeline (Syllable Assembler and Guru/Laghu Classifier) is architecturally identical; only the Unicode codepoint sets change from Telugu (U+0C00–U+0C7F) to Kannada (U+0C80–U+0CFF). The five guru/laghu classification rules (Section 9.3) apply unchanged.

10.2 Gaṇa NFA for Ragale

The Ragale Gaṇa NFA validates each line against the regular pattern:

$$L_{\text{line}} = (\text{III} \mid \text{IIU})^3 \cdot \text{IIU} \tag{2}$$

Table 10: Dvipada vs. Ragale constraint comparison.

Aspect	**Dvipada (Telugu)**	**Ragale (Kannada)**
Gaṇa patterns	8 (6 Indra + 2 Surya)	2 (III, IIU)
Formal language	$(6 \text{ pat.})^3 \cdot (2 \text{ pat.})$	$(\text{III} \mid \text{IIU})^3 \cdot \text{IIU}$
Syllables/line	11–15 (variable)	12 (fixed)
Gaṇa sizes	2–4 syllables	3 (uniform)
Prāsa	7 states, 3 equiv. groups	7 states, 2 equiv. groups
Yati	4 states (optional)	Not used
Guru ending	Implicit	Enforced (gaṇa 4 = IIU)

Only two gaṇa patterns are permitted (Table 2): III (three *laghu*) at positions 1–3, and IIU (two *laghu* + one *guru*) at all four positions. Position 4 is constrained to IIU, structurally enforcing the *guru* ending rule. This yields exactly $2^3 = 8$ valid line patterns, all of length 12 syllables.

The NFA has **22 states**: slots 0–2 each spawn 2 candidate ganas with 3 sub-positions ($2 \times 3 \times 3 = 18$), slot 3 spawns only IIU ($1 \times 3 = 3$), plus 1 ACCEPT state. Compared to the Dvipada Gaṇa NFA (69 states), the Ragale variant achieves a $3.1\times$ reduction.

Dead state detection (Algorithm 1) simplifies because all ganas are exactly 3 syllables: $\mathit{min_dist}(s) = \mathit{max_dist}(s) = (3 - j) + (3 - s) \times 3$, where j is the sub-position and s is the slot index.

10.3 Prāsa and Guru Ending

The Prāsa NFA retains the same 7-state structure (LINE1_SYL0 through ACCEPT/REJECT) with two Kannada-specific equivalence groups: laterals {ḷa ↔ la} and sibilants {śa ↔ ṣa ↔ sa}. The rhotic group ({ṛa, ra}) present in Telugu is absent in Kannada, reducing from 3 to 2 equivalence pairs.

The *guru* ending constraint is not checked as a separate rule; it is structurally guaranteed by the Gaṇa NFA's restriction of position 4 to IIU, whose final syllable is always *guru* (U).

10.4 State Budget

Table 11 shows the Ragale state budget.

The total state count is 35 ($2.3\times$ fewer than Dvipada's $\sim$80). The product automaton is $22 \times 7 = 154$ states ($12.5\times$ fewer than Dvipada's 1,932), reflecting

Table 11: State budget for the Ragale constrained decoding system.

Component	Type	States
Codepoint Classifier	Stateless	0 (inlined)
Syllable Assembler	Mealy FST	4
Guru/Laghu Classifier	Mealy FST	2
Position Tracker	Controller	3 (counter)
Gaṇa NFA	NFA	22
Prāsa NFA	NFA	7
Yati NFA	—	0 (not used)
Total (separate)		**35**
Product (equiv.)	DFA	154

the reduced combinatorial complexity of the 2-pattern gaṇa system. Using separate NFAs still provides a $4.4\times$ reduction over the product (35 vs. 154).

10.5 Decoding Strategies

All three decoding strategies from Section 9 apply to Ragale without algorithmic modification:

- **Rejection sampling** (Algorithm 4): ANALYZETEXT (Algorithm 2) runs the Kannada FST pipeline with VALID_LINE_LENGTHS $= \{12\}$.
- **Logit masking with backtracking** (Algorithm 5): BUILDGANAMASK (Algorithm 3) tests each Kannada token against the Ragale COMPOSITESTATE, which checks gaṇa reachability and prāsa validity (no yati check).
- **Hybrid** (Algorithm 6): identical mask-then-reject loop.

The only implementation change is the reduced gaṇa pool (2 patterns vs. 8) and the removal of the yati alive check from COMPOSITESTATE.IS_ALIVE(). Dead state detection (Algorithm 1) uses simplified closed-form bounds where *min_dist* $=$ *max_dist* (all ganas are 3 syllables). At runtime, the smaller NFA produces fewer branches, reducing the per-step mask computation cost.

11 End-to-End Evaluation

We evaluate the symbolic enforcer end-to-end on both metres (*dvipada*, *utsaha ragale*) by running every combination of two models and three decoding strategies,

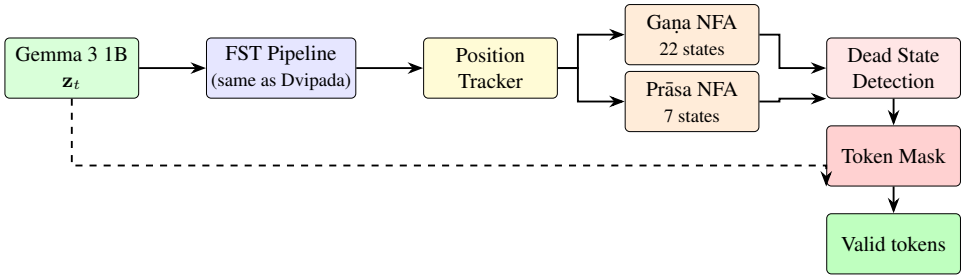

Figure 3: Ragale Symbolic Enforcer. Compared to the Dvipada pipeline (Figure 1), Yati NFA is removed and the Gaṇa NFA uses 22 states (2 patterns) instead of 69 (8 patterns). The FST pipeline is identical except for Kannada Unicode codepoints.

for a total of twelve benchmark configurations.

Setup. For each language we test the base instruction-tuned model (`google/gemma-3-1b-it`) and its LoRA fine-tuned variant (merged for Telugu, adapter for Kannada) under the three decoding strategies introduced in Section 9: rejection-free *masking only*, *masking with checkpoint backtracking*, and the *hybrid* mask + NFA rejection sampler. Each (model, method) cell generates 102 poems from 3 prompts $\times$ 34 seeds (seeds $42 + 7k$ for $k \in [0, 33]$), giving **1,224 total generations** across the matrix. Sampling uses temperature $\tau = 0.7$, top-$p = 0.9$, and `max_new_tokens` $= 150$. All runs use a single NVIDIA RTX 5050 laptop GPU with bfloat16 weights (no quantisation).

Prompts. Telugu prompts are mythological/devotional themes drawn from the corpus ("*tallI prEma annintikaMTe goppadi*", "*hanumaMtuDu samudramunu dATi laMkanu cErenu*", "*SrIrAmuDu sItAdEvini rakshiMcuTaku laMkaku veLLenu*"). Kannada prompts are short topical themes (*Bubbles*, *Moon*, *Rain*) instantiated against the prompt templates of Section 9: a Kannada-system prompt for the base model and an English instruction-tuning template that matches the LoRA training format.

Metrics. We report (i) *poem accuracy*, the fraction of generations whose *both* lines pass the FST+NFA validator; (ii) *line accuracy*, the fraction of individual lines that pass; (iii) average wall time per poem; (iv) average tokens generated; and (v) average backtrack count. Validation uses the same chandas pipeline that filtered the training corpus in Section 6, so generation and dataset construction are scored against an identical specification.

Table 12: Dwipada end-to-end constrained decoding results ($n = 102$ poems per row; 3 prompts $\times$ 34 seeds). Hybrid = masking + NFA rejection sampling.

Model	Method	Valid	Poem %	Line %	s/poem
gemma3-1b-base	masking_only	32/102	31.4	61.8	3.49
gemma3-1b-base	masking_backtrack	93/102	91.2	95.6	3.45
gemma3-1b-base	hybrid	102/102	**100.0**	100.0	1.99
gemma3-1b-merged	masking_only	37/102	36.3	67.6	2.10
gemma3-1b-merged	masking_backtrack	80/102	78.4	89.2	2.65
gemma3-1b-merged	hybrid	102/102	**100.0**	100.0	1.53

Table 13: Ragale end-to-end constrained decoding results ($n = 102$ poems per row; 3 prompts $\times$ 34 seeds). Hybrid = masking + NFA rejection sampling.

Model	Method	Valid	Poem %	Line %	s/poem
gemma3-1b-base	masking_only	13/102	12.7	37.0	10.71
gemma3-1b-base	masking_backtrack	13/102	12.7	37.0	3.78
gemma3-1b-base	hybrid	69/102	67.6	80.2	1.33
gemma3-1b-lora	masking_only	98/102	96.1	96.6	1.42
gemma3-1b-lora	masking_backtrack	98/102	96.1	96.6	1.43
gemma3-1b-lora	hybrid	102/102	**100.0**	100.0	2.03

Reproducibility. All twelve benchmark scripts and the corresponding result JSON files are distributed under `domino/` (Telugu) and `ragale_pipeline/ragale_inference_scripts/` (Kannada) in the project repository.

12 Results

Tables 12 and 13 report poem-level accuracy for the twelve configurations. Per-topic breakdowns and efficiency statistics are deferred to Appendix E.

Strategy comparison. Hybrid is the only strategy that reaches 100% poem accuracy across every fine-tuned configuration we tested (Dwipada base, Dwipada merged, Ragale-LoRA), and additionally on Dwipada-base. This confirms the design intent: NFA rejection on top of masking guarantees forward progress toward an ACCEPT state, whereas masking alone only guarantees that the *next* token keeps the NFA reachable. The per-topic breakdowns in Tables 25 and 26 show that the hybrid wins are uniform across prompts, not driven by a single easy topic.

Effect of fine-tuning. The two languages reveal very different fine-tuning sensitivities. For Ragale, LoRA dramatically improves the simplest strategy (masking_only: $12.7\% \rightarrow 96.1\%$, a $7.6\times$ gain), bringing it within four points of full validity without any rejection or backtracking. For Dwipada the same comparison is much smaller ($31.4\% \rightarrow 36.3\%$), but the merged model still reaches 100% under hybrid with the lowest wall time of any configuration (1.53 s/poem). Fine-tuning therefore collapses most of the search burden into the model's prior, leaving the decoder mostly to verify rather than reject.

Backtracking is asymmetric across languages. For Dwipada-base, checkpoint backtracking lifts validity from 31.4% to 91.2% (a $2.9\times$ gain), confirming that the larger Dwipada gana space (8 patterns, 11–15 syllables) admits enough alternative completions for retry to succeed. For Ragale, masking_only and masking_backtrack produce *identical* validity numbers (12.7% on base, 96.1% on LoRA); backtracking only changes runtime (Ragale-base: $1093 \rightarrow 386$ s, since backtracking escapes earlier from doomed sequences). This is consistent with the smaller Ragale gana space (2 patterns, fixed 12 syllables): once the model drifts off, the constrained alphabet leaves no alternative path, so backtracking cannot find a different valid completion. Notably, on Dwipada-merged, backtracking actually *hurts* accuracy ($36.3\% \rightarrow 78.4\%$ where the same model reaches 100% under hybrid), because retries re-roll the model into the same low-probability tail it was already in.

Cost of constraints. Average tokens per poem decreases monotonically with constraint strength on the Dwipada base model ($52.1 \rightarrow 34.0 \rightarrow 29.9$ across the three methods), and hybrid has the lowest wall time on the LoRA models despite doing the most per-token work. Stronger constraints commit the model to valid syllables earlier, eliminating the wasted generation that the weaker strategies spend exploring dead ends. Appendix Table 27 reports the full per-poem efficiency profile.

Failure modes. Ragale-base failures concentrate in tokens that resemble phonetic transliterations of the English topic name, e.g. *babbis* for "Bubbles" or *baDDis* for "Birds". The base model defaults to echoing the prompt in Kannada script, producing fragments that fail syllable count and prasa simultaneously. Fine-tuning eliminates this mode entirely, which explains the steep $12.7\% \rightarrow 96.1\%$ jump under masking_only.

13 Conclusion

We have presented IndicNeuroSym, a complete neuro-symbolic pipeline for classical Dravidian metrical poetry generation, instantiated on Telugu *dvipada* and Kannada *utsaha ragale*. Our four contributions are: (i) a 27,881-couplet validated Dwipada corpus and 1,012 synthetic Ragale poems, both at 100% prosodic integrity and 97.0% mean LLM-judge score; (ii) an instruction fine-tuning pipeline that produces a LoRA adapter per language; (iii) a chained FST + parallel NFA decoder using $\sim$80 states for Dwipada and $\sim$35 for Ragale, a $24\times$ and $4.4\times$ reduction over the corresponding product automata; and (iv) 1,224 end-to-end constrained generations across 12 configurations benchmarking three decoding strategies on two models per language.

The headline finding is that the hybrid (mask + NFA rejection) strategy reaches **100% poem accuracy** on Dwipada with both base and merged models and on Ragale with the LoRA model, while being the *fastest* strategy on every fine-tuned configuration. LoRA fine-tuning lifts Ragale masking-only from 12.7% to 96.1%, demonstrating that even modest adapter training collapses most of the symbolic search burden into the model prior. The same architecture transfers between the two languages with only structural simplifications (no Yati NFA, smaller Gana NFA), no algorithmic changes to the three decoding strategies in Appendix G.

Future work. Six concrete extensions follow naturally from the present pipeline. First, we plan to involve subject-matter experts (classical Telugu and Kannada scholars) to manually annotate, correct, and rate a stratified subsample of the corpus, using the resulting feedback both to refine the LLM-as-a-Judge rubric of Section 6 and to establish a human-validated gold subset for downstream evaluation. Second, building on the curation methodology of Section 5, we intend to assemble a broader classical Telugu literary corpus that spans additional metrical forms beyond *dvipada*, with the same four-level validation pipeline applied uniformly. Third, we aim to generalise the framework into a language-agnostic toolkit: the FST+NFA pipeline already separates language-specific Unicode codepoint sets (Section 9.3) from metre-specific NFA topology, and exposing these as configurable modules would allow rapid onboarding of additional Indic and non-Indic scripts. Fourth, we plan to evaluate the system under a syllable-aware tokenizer paired with BPE: the current Gemma 3 BPE fragments *akṣaralu* in ways that misalign with prosodic syllable boundaries (cf. Section 1), and a tokenizer that respects *akṣara* units should reduce the dynamic mask burden and improve generation fluency for agglutinative Dravidian languages. Fifth, extending the architecture to additional prosodic metres (Kannada *ṣaṭpadi*, Telugu *sīsa*, and Hindi/Marathi *chand* traditions) is straightforward in principle, since the same chained-FST +

parallel-NFA decomposition should apply wherever syllable weight, foot patterns, and rhyme can be expressed in regular languages. Finally, we plan to integrate lexical resources (a Telugu/Kannada thesaurus and inflected verb-form tables) directly into the constrained decoder, so that token masking can prefer semantically and morphologically appropriate continuations rather than relying on the language model alone for word choice; this should yield poems that are not only metrically valid but also lexically coherent and meaning-faithful to the input prompt.

References

C. Kranti and S. Vajjala, "METRICALARGS: A Taxonomy for Studying Metrical Poetry with LLMs," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

Pawan Goyal, Sayantan Adak, and Vatsal Gupta, "Chandomitra: A System for Sanskrit Metrical Poetry Generation," in *Proceedings of the 19th International Conference on Natural Language Processing (ICON)*, pages 145–154, 2022.

Fangxiaoyu Feng, Yinfei Yang, Daniel Cer, Naveen Arivazhagan, and Wei Wang, "Language-Agnostic BERT Sentence Embedding," *arXiv preprint arXiv:2007.01852*, 2022.

Nils Reimers and Iryna Gurevych. 2020. Making Monolingual Sentence Embeddings Multilingual Using Knowledge Distillation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 4512–4525.

Samruddhi Deode, Janhavi Gadre, Aditi Kajale, Ananya Joshi, and Raviraj Joshi, "L3Cube-IndicSBERT: A simple approach for learning cross-lingual sentence representations using multilingual BERT," *arXiv preprint arXiv:2304.11434*, 2023.

Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu, "G-Eval: NLG Evaluation Using GPT-4 with Better Human Alignment," *arXiv preprint arXiv:2303.16634*, 2023.

Tom Kocmi and Christian Federmann, "Large Language Models Are State-of-the-Art Evaluators of Translation Quality," *Proceedings of the 24th Annual Conference of the European Association for Machine Translation*, pp. 193–203, 2023.

Brandon T. Willard and Rémi Louf, "Efficient Guided Generation for Large Language Models," *arXiv preprint arXiv:2307.09702*, 2023.

Luca Beurer-Kellner, Marc Fischer, and Martin Vechev, "Prompting Is Programming: A Query Language for Large Language Models," *Proceedings of the ACM on Programming Languages (PLDI)*, vol. 7, 2023.

Guidance-AI, "Guidance: A Guidance Language for Controlling Large Language Models," `https://github.com/guidance-ai/guidance`, 2023.

Sreenivas Nidamarthy, "Comparative Study of Kannada and Telugu Consonants, Consonant Sub Forms, Vowels, Signs," *Shikshan Sanshodhan: Journal of Arts, Humanities and Social Sciences*, vol. 4, no. 9, 2021.

Chirravuri Srirama Murthy, "Chandodarpanamu," Digitised text at `https://archive.org/details/chandodarpanamu018848mbp`, 1848.

Gemma Team, "Gemma: Open Models Based on Gemini Research and Technology," *arXiv preprint arXiv:2403.08295*, 2024.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen, "LoRA: Low-Rank Adaptation of Large Language Models," *International Conference on Learning Representations (ICLR)*, 2022.

Rico Sennrich, Barry Haddow, and Alexandra Birch, "Neural Machine Translation of Rare Words with Subword Units," *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 1715–1725, 2016.

Michael A. Covington and Joe D. McFall. 2010. Cutting the Gordian Knot: The Moving-Average Type–Token Ratio (MATTR). *Journal of Quantitative Linguistics*, 17(2):94–100.

George Udny Yule. 1944. *The Statistical Study of Literary Vocabulary*. Cambridge University Press.

Anthony Honoré. 1979. Some Simple Measures of Richness of Vocabulary. *Association for Literary and Linguistic Computing (ALLC) Bulletin*, 7(2):172–177.

Herbert S. Sichel. 1975. On a Distribution Law for Word Frequencies. *Journal of the American Statistical Association*, 70(351):542–547.

A Complete *Dvipada* Prosodic Rules

This appendix provides complete rule tables for the *dvipada* prosodic framework summarised in Section 3.

A.1 Guru-Laghu Classification Rules

Table 14 lists the five conditions under which a syllable is classified as *guru* (U). A syllable not meeting any condition defaults to *laghu* (I). Rule 5 is subject to the word-boundary constraint: it fires when the next syllable begins a conjunct cluster *within the same word* but is blocked by an intervening space; newlines are transparent.

Table 14: Guru-laghu classification rules from *Chandhodarpanamu* (Chirravuri Srirama Murthy, 1998).

R.	Name	Condition
1	Long vowel	Syllable contains aa, ii, uu, ee, oo, ai, or au
2	Diphthong	Vowel is ai or au
3	Anusvara / Visarga	Syllable ends in nasalisation (ṃ) or aspiration (h)
4	Pollu	Syllable ends in a virama (bare consonant, no following vowel)
5	Conjunct position	Next syllable (same word) begins with a conjunct cluster

Exceptions retained as laghu:
(a) Syllables containing the vocalic ṛ/Ṝ and the immediately preceding syllable.
(b) Syllables preceding a "ra-vattu" (ra-conjunct) in many poetic contexts.

A.2 *Prāsa* Equivalence Groups

Table 15 lists the three phonemic equivalence groups for *prāsa* consonant matching. All other consonants (approximately 30) form singleton equivalence classes.

Table 15: *Prāsa* consonant equivalence groups.

Group	Members (IAST)	Phonetic Basis
1	{la, ḷa}	Laterals
2	{śa, sa}	Sibilants
3	{ṛa, ra}	Rhotics

A.3 *Yati Maitri* Groups

Table 16 lists the 11 *maitri* equivalence groups for *yati* alliteration checking. Consonants and vowels within a group are treated as harmonically equivalent.

Table 16: *Yati maitri* groups (IAST romanisation).

#	Members (IAST)	Phonetic Basis
1	a, aa, ai, au, ha, ya, aṃ, ah	Open vowels + glides
2	i, ii, e, ee, ṛ	Front vowels
3	u, uu, o, oo	Back vowels
4	ka, kha, ga, gha, kṣa	Velars
5	ca, cha, ja, jha, śa, ṣa, sa	Palatals + sibilants
6	ṭa, ṭha, ḍa, ḍha	Retroflexes
7	ta, tha, da, dha	Dentals
8	pa, pha, ba, bha, va	Labials
9	ra, la, ṛa, ḷa	Liquids
10	na, ṇa	Nasals
11	ma	Labial nasal

B FST Pipeline Details

B.1 Codepoint Classifier Output Categories

Table 17 describes the 11 character categories output by Stage 1 of the FST pipeline.

Table 17: Codepoint classifier output categories (Telugu Unicode block, U+0C00–U+0C7F).

Category	**Sym.**	**Range**	**Description**
CONSONANT	C	U+0C15–39	ka to Ra
INDEP. VOWEL	V	U+0C05–14	a to au
MATRA	M	U+0C3E–4C	Vowel signs
VIRAMA	V^{-}	U+0C4D	Halant
ANUSVARA	A	U+0C02	ṃ
VISARGA	VS	U+0C03	ḥ
AI LENGTH	AL	U+0C56	ai-length mark
SPACE	SP	U+0020	Space
NEWLINE	NL	U+000A	Newline
ZWNJ	ZW	U+200C	Zero-width NJ
OTHER	OT	(other)	Digits, punctuation

B.2 Guru/Laghu Classifier FST Transition Table

Table 18 gives the complete transition function of Stage 3. S denotes an incoming syllable; P is the buffered pending syllable; $S2$ is the lookahead syllable. PEND. CLEAR and PEND. BOUNDARY abbreviate PENDING_CLEAR and PENDING_BOUNDARY.

Table 18: Guru/Laghu classifier FST transition table.

From	Input	Action	Next
IDLE	syllable S	buffer S; compute self_class(S)	Pend. Clear
IDLE	space / NL	emit boundary	IDLE
Pend. Clear	syllable $S2$	if conjunct: emit U (R5); else emit self_class(P); buffer $S2$	Pend. Clear
Pend. Clear	space	set boundary flag	Pend. Boundary
Pend. Boundary	syllable $S2$	emit self_class(P); emit boundaries; buffer $S2$	Pend. Clear
Pend. Boundary	space / NL	buffer boundary	Pend. Boundary
Pend. Clear	FLUSH	emit self_class(P)	IDLE
Pend. Boundary	FLUSH	emit self_class(P); emit boundaries	IDLE

C Detailed Validation Results

C.1 Length Ratio Distribution

Table 19 reports the full distribution of length ratios $r = |P|/|V|$ between Telugu prose meaning and poetic verse.

Table 19: Length ratio distribution ($n = 29{,}343$). Mean ratio: 1.64; median: 1.61. Enforced bounds: $0.8 \leq r \leq 2.5$.

Ratio Range	Count	Share
0.50 – 0.80	1	<0.1%
0.80 – 1.00	78	0.3%
1.00 – 1.50	9,698	33.1%
1.50 – 2.00	16,371	55.8%
2.00 – 5.11	3,195	10.9%
Pass rate	**99.1% (29,085 / 27,881)**	

C.2 Extended Embedding Model Results

Tables 20 and 21 provide full per-pair results for mSBERT-mpnet (**?**) and L3Cube-IndicSBERT (Deode et al., 2023) respectively.

Table 20: mSBERT-mpnet results across all text pairs.

Pair	Pass%	Mean	Median
Poem – Telugu meaning	63.7	0.685	0.701
Poem – English meaning	4.4	0.424	0.422
Telugu – English	23.7	0.501	0.515

Table 21: L3Cube-IndicSBERT results across all text pairs.

Pair	Pass%	Mean	Median
Poem – Telugu meaning	40.1	0.614	0.618
Poem – English meaning	0.0	0.255	0.251
Telugu – English	21.0	0.579	0.585

C.3 LaBSE Similarity Distribution

Table 22 gives the full distribution of LaBSE Telugu-to-English cosine similarities.

Table 22: LaBSE Telugu meaning – English meaning similarity distribution ($n = 29{,}343$).

Range	Count	Share
0.00 – 0.30	3	<0.1%
0.30 – 0.50	592	2.0%
0.50 – 0.70	10,620	36.2%
0.70 – 0.85	16,294	55.5%
0.85 – 1.00	1,834	6.3%

C.4 LLM-as-a-Judge Aggregate Score Distribution

Table 23 gives the full distribution of aggregate scores (out of 25). Table 24 gives the stratified source breakdown.

Table 23: LLM-as-a-Judge aggregate score distribution ($n = 250$). Mean: 24.25/25 (97.0%).

Score / 25	Count	Share
25	172	68.8%
24	33	13.2%
23	22	8.8%
22	8	3.2%
21	6	2.4%
20	1	0.4%
19	2	0.8%
18	4	1.6%
17	1	0.4%
16	1	0.4%

Table 24: LLM-as-a-Judge stratified sample source distribution.

Source	Samples
Ranganatha Ramayanamu	42
Basavapuranamu	42
Dvipada Bhagavatamu 2	42
Srirama Parinayamu	42
Palanati Vira Charitra	41
Synthetic (LLM)	41
Total	**250**

D Detailed Baseline Generation Examples

D.1 Gemma-3-1B Base Model

The base 1B model generates grammatically plausible but prosodically invalid Telugu across all five prompts (0% valid). Failures include single-line outputs and consistent near-random *prāsa* matching (20.0% average). The model shows no evidence of having internalised *dvipada* structure from pretraining alone.

D.2 Gemma-3-1B LoRA Fine-Tuned Model

Three of five prompted generations were fully valid (80% valid). *Prāsa* was perfect across all five runs (100.0%). Example of a fully valid generation (Prompt 2):

naajilo janudaina naahaaravatula
gaajilo janiyinci kadubhaktitoda

Both lines open with "na-" (satisfying *yati*); the second syllables of each line share the same consonant equivalence class (satisfying *prāsa*); and all four feet in each line follow valid *gaṇa* patterns.

D.3 Gemma-3-4B Base Model

The 4B base model (20% valid) demonstrates partial metre awareness. Prompt 3 produced a valid couplet by outputting gana-name sequences (e.g., "nala nala nala naga ..."), suggesting the model has encountered *dvipada* structural descriptions in pretraining data. Prompts 2 and 5 produced invalid or single-line outputs. The model's score of 50.0% reflects partially correct *gaṇa* patterns even when full validity is not achieved.

E Detailed End-to-End Benchmark Results

This appendix provides per-topic accuracy and per-poem efficiency statistics for the twelve constrained-decoding configurations summarised in Section 12 (Tables 12 and 13). All numbers are extracted directly from the benchmark JSON files distributed with the project repository.

E.1 Per-topic Dwipada accuracy

Table 25 reports the number of valid poems out of 34 seeds for each Telugu prompt. T1 = "*tallI prEma annintikaMTe goppadi*" (mother's love), T2 =

"*hanumaMtuDu samudramunu dATi laMkanu cErenu*" (Hanuman crosses to Lanka), T3 = "*SrIrAmuDu sItAdEvini rakshiMcuTaku laMkaku veLLenu*" (Rama goes to Lanka). Each cell shows valid/34 ($p\%$).

Table 25: Per-topic poem accuracy on Dwipada (3 prompts $\times$ 34 seeds = 102 poems per row).

Model	**Method**	**T1**	**T2**	**T3**
gemma3-1b-base	masking_only	11/34 (32.4)	14/34 (41.2)	7/34 (20.6)
gemma3-1b-base	masking_backtrack	32/34 (94.1)	31/34 (91.2)	30/34 (88.2)
gemma3-1b-base	hybrid	34/34 (100.0)	34/34 (100.0)	34/34 (100.0)
gemma3-1b-merged	masking_only	12/34 (35.3)	9/34 (26.5)	16/34 (47.1)
gemma3-1b-merged	masking_backtrack	27/34 (79.4)	28/34 (82.4)	25/34 (73.5)
gemma3-1b-merged	hybrid	34/34 (100.0)	34/34 (100.0)	34/34 (100.0)

E.2 Per-topic Ragale accuracy

Table 26 reports the same breakdown for the three Kannada topics.

Table 26: Per-topic poem accuracy on Ragale (3 prompts $\times$ 34 seeds = 102 poems per row).

Model	**Method**	**Bubbles**	**Moon**	**Rain**
gemma3-1b-base	masking_only	4/34 (11.8)	4/34 (11.8)	5/34 (14.7)
gemma3-1b-base	masking_backtrack	4/34 (11.8)	4/34 (11.8)	5/34 (14.7)
gemma3-1b-base	hybrid	17/34 (50.0)	24/34 (70.6)	28/34 (82.4)
gemma3-1b-lora	masking_only	33/34 (97.1)	34/34 (100.0)	31/34 (91.2)
gemma3-1b-lora	masking_backtrack	33/34 (97.1)	34/34 (100.0)	31/34 (91.2)
gemma3-1b-lora	hybrid	34/34 (100.0)	34/34 (100.0)	34/34 (100.0)

E.3 Per-poem efficiency statistics

Table 27 reports the per-poem cost profile across all twelve configurations: average tokens generated, average mask computations, average backtracks, and total wall time over 102 poems.

Table 27: Per-poem efficiency profile across the twelve benchmark configurations. Tok = mean tokens generated; Mask = mean dynamic NFA mask computations; BT = mean backtrack count; Total = total wall time over 102 poems.

Lang	Model	Method	Tok	Mask	BT	Total (s)
Dwipada	gemma3-1b-base	masking_only	52.1	50.2	0.00	355.7
Dwipada	gemma3-1b-base	masking_backtrack	34.0	49.1	7.10	352.1
Dwipada	gemma3-1b-base	hybrid	29.9	27.9	0.01	202.9
Dwipada	gemma3-1b-merged	masking_only	44.9	38.9	0.00	214.4
Dwipada	gemma3-1b-merged	masking_backtrack	38.9	54.5	9.06	270.4
Dwipada	gemma3-1b-merged	hybrid	34.0	28.0	0.00	156.0
Ragale	gemma3-1b-base	masking_only	77.2	75.7	0.00	1092.9
Ragale	gemma3-1b-base	masking_backtrack	77.2	75.7	0.00	386.0
Ragale	gemma3-1b-base	hybrid	31.1	29.2	0.01	136.0
Ragale	gemma3-1b-lora	masking_only	25.8	23.8	0.00	145.2
Ragale	gemma3-1b-lora	masking_backtrack	25.8	23.8	0.00	145.7
Ragale	gemma3-1b-lora	hybrid	25.7	23.7	0.00	207.1

E.4 Methodology notes

Each row in the tables above corresponds to a single benchmark script invocation. Seeds follow the schedule $42 + 7k$ for $k \in [0, 33]$ to guarantee reproducibility while spanning the full RNG space the sampler explores. The validator is the same chandas pipeline (FST + parallel NFAs) used during corpus construction in Section 6, so a poem labelled *valid* here would also pass the inclusion criterion that gated entry into the training corpus. Prompt templates differ between the base and fine-tuned models (English instruction template for LoRA models, Kannada/Telugu system-prompt template for base models) so that each model is queried in the format closest to its training distribution; both templates are described in Section 9. The Ragale base model is the only configuration where the same script run twice (masking_only vs masking_backtrack) produces identical *validity* numbers but different *wall times*, because backtracking can escape doomed sequences earlier even when no alternative valid completion exists.

F Kannada Script Conversion Details

F.1 Character Coverage

Table 28 summarises the bidirectional converter's character coverage by category, based on Nidamarthy (2021).

Table 28: Kannada-Telugu bidirectional converter character coverage (Nidamarthy, 2021).

Category	Count	Resemblance
Consonants (high)	17	99%
Consonants (medium)	5	70–99%
Consonants (low)	5	<70%
Remaining consonants	8	(varies)
Vowels + signs	33	95–99%
Numerals	10	1:1 mapping
Total	**78+**	

F.2 Output Schema

Each entry in the converted Kannada dataset contains: the original Telugu poem; the converted Kannada poem; Telugu and Kannada prose meanings; the English meaning (unchanged); word-to-word glosses in Kannada script; a conversion quality score; and per-entry counts of high-, medium-, and low-resemblance characters used to compute the quality score.

F.3 Limitations

The conversion operates purely at the character level and cannot handle script-specific ligature conventions or rendering differences between the two scripts. Kannada-specific prosody rules are not yet validated; the converter assumes that *dvipada* metrical rules transfer identically from Telugu to Kannada, which holds for shared Dravidian metres but would require independent verification for Kannada-specific forms. Converted prose meanings may sound unnatural when read as Kannada, though they function adequately as conditioning prompts for generation.

Repo: `https://github.com/samvarankashyap/indicnuerosym`

G Constrained Decoding Algorithms

This appendix presents the formal pseudocode for the dead state detection mechanism (Algorithm 1), the stateless analysis subroutine (Algorithm 2), the dynamic mask builder (Algorithm 3), and the three constrained decoding strategies described in Section 9: rejection sampling (Algorithm 4), logit masking with backtracking (Algorithm 5), and hybrid mask+rejection (Algorithm 6).

Notation. θ: language model; $\mathcal{T}$: tokenizer; $\mathbf{z}$: logit vector; $\boldsymbol{\pi}$: probability distribution; τ: temperature; $\mathbf{y}$: generated token sequence; $\mathcal{S}$: COMPOSITESTATE (incremental FST+NFA); $\mathcal{B}$: active NFA branches; $\mathcal{V}_{\text{tel}}$: pre-extracted Telugu token set; $\mathbf{M}_s$: static mask (0 for Telugu/EOS, $-\infty$ elsewhere); ℓ: lines completed; $\mathcal{C}$: checkpoint ring buffer; b, b_c: total and consecutive backtrack counters.

Algorithm 1 ISREACHABLE: Dead State Detection

Require: $\mathcal{B}$: active NFA branches; n: syllables consumed in current line
Ensure: **true** iff any branch can reach ACCEPT with $n_{\text{total}} \in [11, 15]$

```
for (s, g, j, M) ∈ B do
    if s = ACCEPT then
        if n ∈ {11, ..., 15} then return true; continue
    end if
    r ← |POOL(s)[g]| − j                          ▷ symbols left in current gaṇa
    if s ≤ 2 then                                 ▷ Indra slot
        ℓ_o ← r + (2−s) · 3 + 2                   ▷ min to accept
        h ← r + (2−s) · 4 + 3                     ▷ max to accept
    else                                          ▷ Surya slot (s = 3)
        ℓ_o ← r; h ← r
    end if
    if n + ℓ_o ≤ 15 and n + h ≥ 11 then
        return true
    end if
end for
return false                                      ▷ all branches doomed
```

Algorithm 2 ANALYZETEXT: Stateless FST+NFA Analysis

Require: Unicode string w (prefix-stripped)
Ensure: (alive, ℓ_c, acc, n, $\mathcal{B}$)

```
Init fresh SYLLASSEMBLER A, GURULAGHUCLASSIFIER G
B ← SPAWNSLOT(0, ()); ℓ_c ← 0; mk ← []
for item ∈ A.process(w) do
    if item = "\n" then
        Flush G; advance B with labels
        if ∃ (ACCEPT, ∗, ∗, ∗) ∈ B then
            ℓ_c += 1
        end if
        B ← SPAWNSLOT(0, ()); mk ← []
    else if item = "␣" then
        Flush boundary; advance B
    else
        Feed syllable to G; advance B
    end if
end for
Flush remaining; advance B
n ← |mk|
acc ← ∃ (ACCEPT, ∗, ∗, ∗) ∈ B ∧ n ∈ {11...15}
alive ← ISREACHABLE(B, n) ∨ (ℓ_c ≥ 2)
return (alive, ℓ_c, acc, n, B)
```

Algorithm 3 BUILDGANAMASK: Dynamic NFA Token Mask

Require: σ: COMPOSITESTATE snapshot; $\mathcal{V}_{\text{tel}} = (\text{ids}, \text{texts})$
Ensure: $\mathcal{V}_v$: token IDs keeping all NFAs alive

```
V_v ← ∅
for (tid, txt) ∈ zip(ids, texts) do
    S' ← FromSnapshot(σ)
    S'.feed_token(txt); S'.flush()
    if S'.is_alive() then                    ▷ gaṇa ∧ prāsa ∧ yati
        V_v ← V_v ∪ {tid}
    end if
end for
return V_v
```

Algorithm 4 Strategy 1: Rejection Sampling

Require: $\theta, \mathcal{T}, \tau, K, p$
Ensure: Two-line Dwipada poem text

```
y ← [ ];  ℓ ← 0
while ℓ < 2 do
    π ← softmax(θ(y)/τ);  sort desc
    st ← AnalyzeText(StripPrefix(T.dec(y)))
    ℓ ← st.ℓ_c;  if ℓ ≥ 2 then break
                                                    ▷ Force newline on completion/overshoot
    if st.acc ∧ st.n ∈ {11...15} then
        y ← y ‖ [NL];  continue
    end if
    if st.n > 15 ∧ ¬st.acc then
        y ← y ‖ [NL];  continue
    end if
    K′ ← K;  if st.n ≥ 8 then K′ ← 2K
                                                    ▷ Rejection loop over top-K′
    V_acc ← ∅;  V_alv ← ∅
    for k = 1 to K′ do
        c ← π_k;  if π(c) < 10^−8 then break
        if c = Eos ∧ ℓ ≥ 2 then select c; break
        if ¬IsTelugu(T.dec(c)) then continue
        r ← AnalyzeText(StripPrefix(T.dec(y ‖ [c])))        ▷ full re-analysis
        if ¬r.alive then continue
        if "\n" ∈ T.dec(c) ∧ r.ℓ_c ≤ ℓ then continue
        if r.n > 15 ∧ ¬r.acc then continue
        if r.acc ∧ r.n ∈ {11...15} then
            V_acc ← V_acc ∪ {(c, π(c))}
        end if
        V_alv ← V_alv ∪ {(c, π(c))}
        if |V_acc| ≥ 5 then break
    end for
                                                    ▷ Select from best candidate set
    V ← V_acc if ≠ ∅, else V_alv
    if V ≠ ∅ then
        Top-p filter;  chosen ← weighted sample from V
    else
                                                    ▷ Exhaustive fallback (500 tokens)
        for k = 1 to 500 do
            if IsTelugu(T.dec(π_k)) then
                r ← AnalyzeText(StripPrefix(T.dec(y ‖ [π_k])))
                if r.alive ∧ (r.n ≤ 15 ∨ r.acc) then
                    chosen ← π_k;  break
                end if
            end if
        end for
    end if
    y ← y ‖ [chosen]
end while
return T.dec(y)
```

Algorithm 5 Strategy 2: Logit Masking with Backtracking

Require: $\theta, \mathcal{T}, \mathcal{V}_{\text{tel}}, \mathbf{M}_s, \tau, \tau_{\max}$=1.2, $\Delta\tau$=0.15, D=6
Ensure: Two-line Dwipada poem text

$\mathbf{y} \leftarrow [\,]$; $\mathcal{S} \leftarrow$ new COMPOSITESTATE(); $\ell \leftarrow 0$; $\tau_c \leftarrow \tau$
$\mathcal{C} \leftarrow [\,]$; $b \leftarrow 0$; $b_c \leftarrow 0$; $d \leftarrow 0$ ▷ ring buffer, counters
for $t = 1$ **to** $T_{\max}$ **do**
  if $\ell \geq 2$ **then break**
  end if
  $\mathbf{z} \leftarrow \theta(\mathbf{y})/\tau_c + \mathbf{M}_s$ ▷ static mask
  ▷ *Checkpoint every 3 tokens*
  if $t \bmod 3 = 0$ **then**
    $\mathcal{C}$.push(t, $\mathcal{S}$.snap(), $|\mathbf{y}|$, ℓ, τ_c)
    if $|\mathcal{C}| > 15$ **then** $\mathcal{C} \leftarrow \mathcal{C}[-15:]$
  end if
  ▷ *Force newline on ACCEPT*
  $\mathcal{S}_f \leftarrow$ clone($\mathcal{S}$); $\mathcal{S}_f$.flush()
  if $\mathcal{S}_f$.has_accept() **then**
    $\mathcal{S}$.feed("\n"); $\ell \leftarrow \mathcal{S}.\ell_c$; $\mathbf{y} \leftarrow \mathbf{y} \,\|\, [\text{NL}]$
    $b_c \leftarrow 0$; **continue**
  end if
  ▷ *Build dynamic NFA mask*
  $\sigma \leftarrow \mathcal{S}$.snap(); $\mathcal{V}_v \leftarrow$ BUILDGANAMASK($\sigma, \mathcal{V}_{\text{tel}}$)
  ▷ *Filter spurious newlines*
  if NL $\in \mathcal{V}_v$ **then**
    $\mathcal{S}' \leftarrow$ FROMSNAP(σ); $\mathcal{S}'$.feed("\n")
    if $\mathcal{S}'.\ell_c \leq \ell$ **then**
      $\mathcal{V}_v \leftarrow \mathcal{V}_v \setminus \{\text{NL}\}$
    end if
  end if
  ▷ *Backtrack if dead end*
  if $(|\mathcal{V}_v| < 3 \;\vee\; \mathcal{S}_f.n > 15)$ **and** $|\mathcal{C}| > 0$ **and** $b < 30$ **then**
    $b \mathrel{+}= 1$; $b_c \mathrel{+}= 1$
    Pop $\min(b_c, |\mathcal{C}|)$ from $\mathcal{C}$
    Restore $\mathcal{S}, \mathbf{y}, \ell, t$ from last popped
    $\tau_c \leftarrow \min(\text{ckpt}.\tau + \Delta\tau \cdot b_c,\ \tau_{\max})$ ▷ escalate
    $d \leftarrow D$; SEED(seed $+ b \cdot 1337$); **continue**
  end if
  ▷ *Apply dynamic mask and sample*
  $\mathbf{M}_d[i] \leftarrow 0$ if $i \in \mathcal{V}_v$, else $-\infty$
  $\mathbf{z} \leftarrow \mathbf{z} + \mathbf{M}_d$; $\boldsymbol{\pi} \leftarrow \text{softmax}(\mathbf{z})$
  chosen $\leftarrow$ sample($\boldsymbol{\pi}, p$)
  ▷ *Commit token*
  $\mathbf{y} \leftarrow \mathbf{y} \,\|\, [\text{chosen}]$; $\mathcal{S}$.feed_token($\mathcal{T}$.dec(chosen))
  $\ell \leftarrow \mathcal{S}.\ell_c$; $b_c \leftarrow 0$
  ▷ *Temperature decay*
  if $d > 0$ **then**
    $d \mathrel{-}= 1$; **if** $d = 0$ **then** $\tau_c \leftarrow \tau$
  end if
end for
return $\mathcal{T}$.dec($\mathbf{y}$)

Algorithm 6 Strategy 3: Hybrid (Mask + Rejection)

Require: $\theta, \mathcal{T}, \mathcal{V}_{\text{tel}}, \mathbf{M}_s, \tau, K_r{=}100$
Ensure: Two-line Dwipada poem text

$\mathbf{y} \leftarrow [\,]$; $\mathcal{S} \leftarrow$ new COMPOSITESTATE(); $\ell \leftarrow 0$; $\tau_c \leftarrow \tau$
$\mathcal{C} \leftarrow [\,]$; $b \leftarrow 0$; $b_c \leftarrow 0$
for $t = 1$ **to** $T_{\max}$ **do**
  if $\ell \geq 2$ **then break**
  end if
  $\mathbf{z} \leftarrow \theta(\mathbf{y})/\tau_c + \mathbf{M}_s$
  if $t \bmod 3 = 0$ **then** checkpoint $\mathcal{C}$ (cap 15)
  ▷ *Force newline on ACCEPT*
  $\mathcal{S}_f \leftarrow$ clone($\mathcal{S}$); $\mathcal{S}_f$.flush()
  if $\mathcal{S}_f$.has_accept() **then**
    $\mathcal{S}$.feed("\n"); $\ell \leftarrow \mathcal{S}.\ell_c$; $\mathbf{y} \leftarrow \mathbf{y} \,\|\, [\text{NL}]$; $b_c \leftarrow 0$; **continue**
  end if
  ▷ *Build + apply dynamic NFA mask*
  $\sigma \leftarrow \mathcal{S}$.snap(); $\mathcal{V}_v \leftarrow$ BUILDGANAMASK($\sigma, \mathcal{V}_{\text{tel}}$)
  Filter spurious newlines (as Alg. 5)
  $\mathbf{M}_d[i] \leftarrow 0$ if $i \in \mathcal{V}_v$, else $-\infty$; $\mathbf{z} \leftarrow \mathbf{z} + \mathbf{M}_d$
  $\boldsymbol{\pi} \leftarrow$ softmax($\mathbf{z}$); sort desc
  ▷ *NFA rejection over top-K_r from masked dist.*
  $\mathcal{V}_{\text{acc}} \leftarrow \emptyset$; $\mathcal{V}_{\text{alv}} \leftarrow \emptyset$
  for $k = 1$ **to** K_r **do**
    $c \leftarrow \pi_k$; **if** $\boldsymbol{\pi}(c) < 10^{-8}$ **then break**
    if $c = \text{EOS} \wedge \ell \geq 2$ **then** select c; **break**
    ▷ Clone state, simulate candidate
    $\mathcal{S}' \leftarrow$ FROMSNAP(σ)
    for each ch $\in \mathcal{T}$.dec(c) **do** $\mathcal{S}'$.feed(ch)
    $\mathcal{S}'_f \leftarrow$ clone($\mathcal{S}'$); $\mathcal{S}'_f$.flush()
    if $\neg\mathcal{S}'_f$.alive() **then continue**
    if "\n" $\in \mathcal{T}$.dec(c) $\wedge\ \mathcal{S}'.\ell_c \leq \ell$ **then continue**
    if $\mathcal{S}'_f.n > 15 \wedge \neg\mathcal{S}'_f$.acc() **then continue**
    if $\mathcal{S}'_f$.has_accept() **then**
      $\mathcal{V}_{\text{acc}} \leftarrow \mathcal{V}_{\text{acc}} \cup \{(c, \boldsymbol{\pi}(c))\}$
    end if
    $\mathcal{V}_{\text{alv}} \leftarrow \mathcal{V}_{\text{alv}} \cup \{(c, \boldsymbol{\pi}(c))\}$
    if $|\mathcal{V}_{\text{acc}}| \geq 3$ **then break**
    if $|\mathcal{V}_{\text{alv}}| \geq 10 \wedge \mathcal{V}_{\text{acc}} = \emptyset$ **then break**
  end for
  ▷ *Select or backtrack*
  $\mathcal{V} \leftarrow \mathcal{V}_{\text{acc}}$ if $\neq \emptyset$, else $\mathcal{V}_{\text{alv}}$
  if $\mathcal{V} \neq \emptyset$ **then**
    Normalize; chosen $\leftarrow$ weighted sample
  else if $|\mathcal{C}| > 0 \wedge b < 30$ **then**
    $b \mathrel{+}= 1$; $b_c \mathrel{+}= 1$; pop $\min(b_c, |\mathcal{C}|)$ from $\mathcal{C}$
    Restore $\mathcal{S}, \mathbf{y}, \ell, t$; $\tau_c \leftarrow \min(\text{ckpt}.\tau + \Delta\tau \cdot b_c,\ \tau_{\max})$
    SEED(seed$+b\cdot 1337$); **continue**
  else
    chosen $\leftarrow \pi_1$ ▷ last resort
  end if
  $\mathbf{y} \leftarrow \mathbf{y} \,\|\, [\text{chosen}]$; $\mathcal{S}$.feed_token($\mathcal{T}$.dec(chosen)); $\ell \leftarrow \mathcal{S}.\ell_c$; $b_c \leftarrow 0$
end for
return $\mathcal{T}$.dec($\mathbf{y}$)

Table 29: Comparison of constrained decoding strategies.

Property	**Rejection** (Alg 4)	**Masking** (Alg 5)	**Hybrid** (Alg 6)
State track.	Stateless	Incremental	Incremental
Per-step	$O(KL)$	$O(\|\mathcal{V}_t\|)$	$O(\|\mathcal{V}_t\|+K_r)$
Seq. dep.	Yes	No	No
Recovery	Exhaust.	Backtrack	Backtrack
Temp. adapt.	None	Escalation	Escalation
Constraints	Gaṇa	All 3	All 3